Balatro RL Agent

A Reinforcement Learning Agent that learns to play Balatro using deep learning.

Overview • How It Works • Credits

Overview

This project creates an AI agent that learns to play and beat Balatro by:

- Injecting Lua code into the Love2D game engine using the Lovely Injector mod system
- Monitoring game state in real-time through direct memory access (money, chips, hands remaining, discards remaining, current hand cards)
- Converting game state into normalized feature vectors to feed into the neural network
- Using reinforcement learning (PPO - Proximal Policy Optimization) to learn optimal card selection and play strategies
- Executing actions through a bidirectional file-based communication protocol (JSON commands for playing or discarding cards)

The agent observes the game through a state vector that encodes game phase, player resources (money, chips, blind target), remaining hands and discards, and the current hand composition (card values, suits, and positions). This normalized state vector is fed into the neural network to determine which cards to play or discard to maximize score and progress through rounds.

The RL model focuses specifically on learning optimal hand play strategies within antes—maximizing hand value by selecting which cards to play or discard each turn. It does not handle inter-round decisions such as selecting power-ups or shop purchases, concentrating instead on mastering the core card selection mechanics that determine success within each ante.

How It Works

- Initial Setup:

I am using the steam version of the game so follow the Lovely Injector instructions first. After that set the launch options in steam as --dump-all.

Essentially what this does is it uses the Lovely Injector to dump all the game files and then we can search them to see how game info is handled. To find your dump folder on **Windows** go to

AppData\Roaming\Balatro\Mods\lovely\dump. Since the dump contains the game files we can just open this into VS Code and do *Ctrl+Shift+F* or *Cmd+Shift+F* to search through all the game files at once to see stuff like: 1. How score is handled, 2. What happens when you play a hand, etc.

Once you have openned the **dump** folder in VS Code, you will see a bunch of Lua files. These are all the important game files that show how logic is handled.

We will make a Symlink between the bridge folder here and the bridge folder in the Lovely Mods directory.

The Bridge: Communication Layer

The bridge is a **Lua-based communication layer** that enables bidirectional communication between the PPO agent and the Balatro game. It acts as an intermediary, allowing the agent to observe game state and execute actions within the game.

What the Bridge Does

The bridge (`bridge/main.lua`) is injected into the game's runtime using the Lovely Injector mod system. It operates by hooking into the game's main update loop and performs the following functions:

1. Game State Monitoring

- Hooks into `Game:update(dt)` to run every frame
- Monitors key game state variables:
  - Current game phase/state (`G.STATE`)
  - Money (`G.GAME.dollars`)
  - Chips (current and blind target)
  - Hands remaining (`G.GAME.current_round.hands_left`)
  - Discards remaining (`G.GAME.current_round.discards_left`)
  - Current hand cards and their properties

2. Event-Based State Snapshotting

The bridge uses an event-driven approach to capture game state only when relevant changes occur. It triggers a state dump when:

- **State Transition**: The game enters the `SELECTING_HAND` state (state 4), indicating the player can make decisions
- **Hand Played**: The number of hands remaining decreases
- **Cards Discarded**: The number of discards remaining decreases
- **Hand Size Changed**: New cards are drawn or cards are removed

When triggered, the bridge outputs a formatted snapshot containing:

- Current phase/state
- Money and chip values
- Hands and discards remaining
- Complete hand information (card indices, values, and suits)

3. Command Execution

The bridge continuously polls for commands from the PPO agent via a JSON file (`command.json`). When a command file is detected:

1. **Reads** the command file containing:
   - `action`: Either `"play"` or `"discard"`
   - `cards`: Array of card indices (1-based) to select
2. **Selects Cards**: Highlights the specified cards in the game's hand
   - Uses `G.hand:unhighlight_all()` to clear previous selections
   - Uses `G.hand:add_to_highlighted(card)` for each specified card index
3. **Executes Action**: Calls the appropriate game function:
   - `G.FUNCS.play_cards_from_highlighted()` for playing cards
   - `G.FUNCS.discard_cards_from_highlighted()` for discarding cards
4. **Cleans Up**: Deletes the command file after processing to prevent re-execution

4. State Tracking

The bridge maintains internal state tracking to detect changes:

- `last_state`: Previous game state value
- `last_hands`: Previous hands remaining count
- `last_discards`: Previous discards remaining count
- `last_card_count`: Previous hand size

This prevents redundant state dumps and ensures the agent only receives updates when meaningful changes occur.

Bridge Configuration

The bridge is loaded into the game via `bridge/lovely.toml`, which configures the Lovely Injector to:

- Patch the game's `main.lua` file
- Inject `require('bridge/main')` after the `require "challenges"` line
- Ensure the bridge code runs alongside the game's main logic

Communication Protocol

The bridge implements a **file-based communication protocol**:

Agent → Game (Actions):

```
PPO Agent writes → command.json → Bridge reads → Bridge executes → Game state
changes
```

Game → Agent (Observations):

```
Game state changes → Bridge detects → Bridge dumps state (console/logs) → Agent
reads
```

The command file format:

```
{
  "action": "play" | "discard",
  "cards": [1, 2, 3]  // 1-based indices
}
```

Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              PPO Agent (Python)                          │
│   ┌──────────────────┐                ┌──────────────────┐               │
│   │  Policy Network  │                │  Value Network   │               │
│   │  (Actor)         │                │  (Critic)        │               │
│   └──────────────────┘                └──────────────────┘               │
│            │                                   │                          │
│            └──────────────────────────────────┘                          │
│                          │                                               │
│                          │                                               │
│                ┌─────────▼─────────┐                                     │
│                │ Action Selector │                                       │
│                └─────────────────┘                                       │
└──────────────────────────┬──────────────────────────────────────────────┘
                           │
                           │ writes command.json
                           │
┌──────────────────────────▼──────────────────────────────────────────────┐
│                         File System                                      │
│                  command.json (temporary)                                │
└──────────────────────────┬──────────────────────────────────────────────┘
                           │
                           │ reads & deletes
                           │
┌──────────────────────────▼──────────────────────────────────────────────┐
│                         Bridge (Lua)                                     │
│   ┌──────────────────────────────────────────────────────────────────┐   │
│   │  Game.update() Hook                                              │   │
│   │   ┌──────────────────┐   ┌──────────────────────────────┐       │   │
│   │   │ check_for_       │   │  State Change Detection      │       │   │
│   │   │ commands()       │   │  - State transitions         │       │   │
│   │   │                  │   │  - Hand/discard changes      │       │   │
│   │   │   • Read JSON    │   │  - Card count changes        │       │   │
│   │   │   • Parse action │   │                              │       │   │
│   │   │   • Select cards │   │   ┌────────────────────┐     │       │   │
│   │   │   • Execute cmd  │   │   │ dump_game_state()  │     │       │   │
│   │   │   • Delete file  │   │   │   • Print state info │   │       │   │
```

```
│  │  └──────────────────┘  │  │  • Format output      │  │   │   │
│  │                        │  └───────────────────────┘  │   │   │
│  └──────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────┘
                          │
                          │
                          │ hooks into & calls game functions
                          │
┌─────────────────────────▼────────────────────────────────────┐
│                     Balatro Game (Love2D)                    │
│  ┌──────────────────────────────────────────────────────┐   │
│  │  Game State (G.*)                                    │   │
│  │  • G.STATE (current phase)                           │   │
│  │  • G.GAME.dollars (money)                            │   │
│  │  • G.GAME.chips (score)                              │   │
│  │  • G.hand.cards[] (current hand)                     │   │
│  │  • G.GAME.current_round (hands/discards left)        │   │
│  │                                                      │   │
│  │  Game Functions                                      │   │
│  │  • G.FUNCS.play_cards_from_highlighted()             │   │
│  │  • G.FUNCS.discard_cards_from_highlighted()          │   │
│  │  • G.hand:add_to_highlighted(card)                   │   │
│  └──────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────┘

Observation Flow (Game → Agent):
Game State → Bridge detects change → Bridge prints/logs state → Agent reads
logs/file

Action Flow (Agent → Game):
Agent decides action → Agent writes command.json → Bridge reads → Bridge selects
cards → Bridge executes → Game updates
```

Key Design Decisions

1. **File-Based Communication**: Uses temporary JSON files instead of sockets/process communication for simplicity and compatibility with the game's Lua runtime
2. **Event-Driven Snapshotting**: Only captures state when meaningful changes occur, reducing noise and improving efficiency
3. **State Tracking**: Maintains previous values to detect transitions rather than polling continuously
4. **Safe Execution**: Uses `pcall()` to wrap state dumps, preventing crashes from unexpected game state
5. **Command Cleanup**: Deletes command files immediately after reading to prevent duplicate executions

Roadmap

- [x] Bidirectional Communication Bridge (Lua/Python)
- [x] State Reflection (Direct Memory Access)
- [x] Feature Encoding (Numerical Vectorization)
- [ ] Imitation Learning (Behavioral Cloning from Human Play)
- [ ] Gymnasium Environment Wrapper

Credits

- Credit to @ethangreen-dev for the Love2D Injector code.
- Credit to Stable-Baseline3 for PPO implementation
- Credit to Gymnasium for the RL environment framework

License

This project is open source and available under the MIT License.